Assignment 3 and 4: CEP

Event-Driven Traffic Alert System

Software Design and Architecture

BSSE 4A and 4B

Course Instructor: Dr. Nida Adnan

Total marks	CLOs covered	Submission
60	**CLO 3 + CLO 4**	**Code + Report + Individual Viva**

Instructions

This is a team assignment. The group should have maximum 3 students. This assignment serves as Complex Engineering Problem (CEP).

Scenario

You are building a simple traffic monitoring system for a city. Traffic cameras send alerts when they detect vehicles, speed violations, or heavy congestion. Your job is to design this system so that:

- Different parts of the system can receive these alerts independently
- Adding a new feature later does not require changing the cameras
- If the same alert is accidentally sent twice, the system does not create a duplicate penalty
- When too many alerts arrive at once, the system handles them safely

The key idea: Event-Driven Architecture

Think of it like a radio broadcast. A radio station (publisher) broadcasts a show. Anyone with a radio (subscriber) can tune in. The station does not know who is listening, and the listeners do not need to call the station to get the show.

In your system:

Traffic camera	=	The radio station (publisher). It broadcasts events like speed violations.
AlertService	=	A listener. It sends penalty notices when it hears a speed violation.
LoggingService	=	Another listener. It records every event for audit purposes.
DashboardService	=	Another listener. It updates the live traffic map.

Event Bus	=	The radio signal carries events from cameras to all listeners.

PART A — CLO 3 Apply Design Patterns (30 marks)

CLO 3 asks you to use the right design pattern for the right reason. You must not just copy a pattern from a textbook; you must explain why it solves a specific problem in this system.

- **Task 1: Build the Event Bus (10 marks)**

Create an EventBus class that sits between the cameras and the services. The cameras call bus.publish(event) and the services call bus.subscribe(). The bus delivers the event to every subscriber.

Your system must support these four event types:

Event type	When is it published?	Who cares about it?
VehicleDetectedEvent	A camera spots a vehicle	Dashboard, Reporting
SpeedViolationEvent	Vehicle exceeds speed limit	Alert, Logging, Reporting
CongestionAlertEvent	Too many vehicles at one intersection	Dashboard, Logging
TrafficClearedEvent	Congestion drops back to normal	Dashboard

To earn full marks, demonstrate that adding a 5th event type requires zero changes to any existing camera code.

- **Task 2: Apply the Observer Pattern (5 marks)**

The Observer Pattern is the detailed design pattern that makes the Event Bus work cleanly. Every subscriber must implement a common interface called IEventSubscriber. The bus stores a list of these interfaces, never a reference to the concrete class (AlertService, LoggingService, etc.).

Required deliverable: A UML class diagram showing IEventSubscriber, EventBus, and all four subscriber classes with correctly labelled arrows.

- **Task 3: Apply the Event Envelope Pattern (5 marks)**

Every event must be wrapped in an EventEnvelope before it travels on the bus. Think of the envelope as the stamp and address on a letter; it tells you who sent it, when, and what version of the format it uses, without opening the letter itself.

Your *EventEnvelope* must contain these fields:

Field	What it is
event_id	A unique ID for this specific event instance (UUID)
correlation_id	Groups related events together (e.g. one vehicle's full journey)
schema_version	Version number of the event format (starts at 1)
source_id	Which camera sent this event
Timestamp	Date and time the event was created
event_type	Name of the event class, e.g. SpeedViolationEvent
Payload	The actual event data

Task 4: Apply the Idempotent Receiver Pattern (10 marks)

Sometimes a network glitch causes the same event to be delivered twice. Without protection, AlertService might send two penalty notices for one speeding vehicle. That is a serious problem.

The Idempotent Receiver Pattern fixes this. Each subscriber checks the event_id of every incoming event. If it has already seen that ID, it ignores the event. If it is new, it processes it and remembers the ID.

Required test (worth marks):

Publish the same SpeedViolationEvent twice (using the same event_id). Then check that AlertService only sent ONE penalty notice, not two.

PART B — CLO 4 Analyze Design Scenarios (30 marks)

CLO 4 is purely written analysis. You do not need to implement the solutions; you need to think about them carefully and write a clear argument. Each scenario is a real-world problem your system might face.

Scenario 1: What happens when the event format changes? (10 marks)

The situation: Six months after your system launches, the city authority wants VehicleDetectedEvent to include a new field: lane_number (which lane the vehicle was detected in). But there are already 200 subscriber instances running on the old format.

Write about:

- **Option A- Backward Compatibility:** Add lane_number as an optional field with a default value. Old subscribers do not break. What are the risks of this approach?
- **Option B- Schema Versioning:** release VehicleDetectedEvent_v2, update schema_version to 2. What happens to subscribers that only understand version 1?
- Which option would you choose, and why? Write a short Architecture Decision Record (ADR) with three parts: the problem, your decision, and what you gain and lose.

Scenario 2: What happens when too many events arrive at once? (10 marks)

The situation: During a football match, all 12 intersections near the stadium send events simultaneously. The bus receives 500 events per second. DashboardService can only process 80 events per second.

Write about:

- **Calculate:** At 500 events/sec in and 80 events/sec out, how many seconds before the queue reaches 10,000 unprocessed events? Show your working.
- **The Bounded Queue tactic:** Set a maximum queue size. When the queue is full, new events must bump out an old one. Implement this in your EventBus.
- **Which event should be dropped**: The oldest one, or the least important one? A CRITICAL congestion alert matters more than a routine vehicle detection. Justify your choice.

Scenario 3: What if only one service fails? (10 marks)

The situation: A SpeedViolationEvent is published. AlertService receives it and sends a penalty notice to the vehicle owner. Then LoggingService crashes before it can write the audit log. A penalty now exists with no record of why it was issued, legally inadmissible.

Write about:

- **Name and explain the problem.** (Hint: it is called the Dual Write Problem.)
- **Describe the Outbox Pattern as a solution: I**nstead of publishing directly to the bus, the camera first writes the event to a local database. A separate background process reads from that database and publishes to the bus. This guarantees the event is never lost.
- **What does the Outbox Pattern cost?** Think about complexity and any extra delay it adds.
- In a traffic enforcement system where penalties must be legally admissible, is this cost worth paying? Write at least 150 words arguing your position.

Marking Rubrics

CLO 3 — Design Pattern Implementation

What you must do	Marks	How you'll be assessed
Build the Event Bus with 4 event types and adding the no change rule highlighted for the 5th event	10	Working code + justification for each event type

What you must do	Marks	How you'll be assessed
Apply the Observer Pattern	5	UML diagram + subscribe/unsubscribe demonstration
Apply the Event Envelope Pattern	5	Envelope class with all 7 fields + written justification
Apply the Idempotent Receiver Pattern	10	Base class + passing test for duplicate event

CLO 4 — Design Scenario Analysis

What you must do	Marks	How you'll be assessed
Scenario 1: Schema evolution, write an ADR	10	Written analysis (min. 200 words) + ADR
Scenario 2: Event flood, bounded queue analysis	10	Calculation + eviction policy + justification
Scenario 3: Dual write, Outbox Pattern analysis	10	Written analysis + domain judgement (min. 150 words)